- Simule todos os trechos de códigos apresentados. Modifique os valores das localizações de memória afetadas para verificar se o funcionamento está coerente.
- Faça um programa que some duas variáveis de 8 bits:

$$C = A + B$$

COM 8 BITS

```
.DSEG
.ORG SRAM_START

A: .BYTE 1;
B: .BYTE 1;
C: .BYTE 1;


.CSEG
 start:

    LDS R16, A
    LDS R17, B

    ADD R16, R17 ; resultado foi guardado no r16

    STS C, R16 ; salvamos no C
```

```
.include "m328Pdef.inc"

.DSEG
.ORG SRAM_START

A: .BYTE 1; A (0x0100): Linha 100, Coluna 00
B: .BYTE 1; B (0x0101): Linha 100, Coluna 01
C: .BYTE 1; C (0x0102): Linha 100, Coluna 02


.CSEG
 start:

    LDS R16, A ; le o valor de A para p R16
    LDS R17, B ; le o valor de B para p R17

    ADD R16, R17 ; soma r16 + r17 e o resultado foi guardado no r16
```

```
    STS C, R16 ; salvamos o resultado no endereço C

loop_final:
    RJMP loop_final ; TRAVA O PROCESSADOR AQUI (evita o erro no simulador)
```

- Refaça o exercício anterior, mas considerando variáveis de 16 bits.
- Observações:
  - Considere as variáveis A, B e C armazenadas sequencialmente a partir do primeiro endereço da memória SRAM.
  - Para o exercício com variáveis de 16 bits, considere a ordenação dos bytes como *little-endian*[1], ou seja, o byte menos significativo é armazenado no menor endereço.

COM 16

```
.DSEG
.ORG SRAM_START

A: .BYTE 2
B: .BYTE 2
C: .BYTE 2


.CSEG
 start:

    LDS R16, A ; AL
    LDS R17, B ; BL

    ADD R16, R17 ;AL + BL

    STS C, R16; CL = AL+ BL

    LDS R16, A+1 ;  AH
    LDS R17, B+1 ; BH

    ADC R16, R17 ; AH + BH + CH

    STS C+1, R16 ; CH = AH + BH + C

——------------------------------------------
.DSEG
```

```
.ORG SRAM_START

A: .BYTE 2
B: .BYTE 2
C: .BYTE 2


.CSEG
 start:

    LDS R16, A ; AL a menos significativo
    LDS R17, B ; BL b menos significativo

    ADD R16, R17 ;AL + BL

    STS C, R16; CL = AL+ BL

    LDS R16, A+1 ;  AH a mais significativo
    ;(Vá até o endereço de A (0x0100), avance 1 posição na memória (0x0101) e busque o byte mais significativo que está guardado l, por isso o +1.)
    LDS R17, B+1 ; BH b mais significativo

    ADC R16, R17 ; AH + BH + CH
    ; soma o carry gerado do ADD anterior, por isso é ADC

    STS C+1, R16 ; CH = AH + BH + C

loop_final:
    RJMP loop_final ; TRAVA O PROCESSADOR AQUI (evita o erro no simulador)
```

COM 16 USANDO REGISTRADORES DIFERENTES:

```
.DSEG
.ORG SRAM_START

A: .BYTE 2
B: .BYTE 2
C: .BYTE 2


.CSEG
 start:

    LDS R16, A ; AL
    LDS R17, B ; BL
```

```
    LDS R18, A+1; AH
    LDS R19, B+1 ; BH

    ADD R16, R17 ;AL + BL
    ASC R18, R19

    STS C, R16; CL = AL+ BL
    STS C+1 , R18

    rjmp start
```

```
.DSEG
.ORG SRAM_START

A: .BYTE 2
B: .BYTE 2
C: .BYTE 2


.CSEG
 start:

    ; 1. Carrega A e B inteiros nos registradores
    LDS R16, A ; AL
    LDS R17, B ; BL
    LDS R18, A+1; AH
    LDS R19, B+1 ; BH


    ; 2. Faz a soma de 16 bits
    ADD R16, R17 ;AL + BL
    ;(Soma byte baixo: AL + BL (Resultado em R16)
    ADC R18, R19
    ;(Soma byte alto com Carry: AH + BH + C (Resultado em R18)


    ; 3. Salva o resultado na variável C da SRAM
    STS C, R16; CL = AL+ BL
    ;(Salva byte baixo no primeiro endereço de C)
    STS C+1 , R18 ; CH = AH + BH
    ;(Salva byte alto no segundo endereço de C)

    rjmp start
```

8 bits = 1 byte (. BYTE 1)

16 bits = 2 bytes (.BYTE 2)

32 bits = 4 bytes (.BYTE 4)

- Faça um programa que some duas variáveis (A e B) e armazena o resultado em uma terceira (C):

```
C = A + B
```

- Observações:
  - Considere todas as variáveis de 32 bits.
  - Considere as varáveis A, B e C armazenadas sequencialmente a partir do primeiro endereço da memória SRAM.
  - Faça 4 implementações utilizando os seguintes modos de endereçamento: Direto, Indireto (sem incrementar), Indireto com Pós-incremento e Indireto com Deslocamento.

Entrega

- Arquivo texto (*.asm) com todos os 4 códigos.

endereçamento direto:

```
.DSEG
.ORG SRAM_START

A: .BYTE 4
B: .BYTE 4
C: .BYTE 4


.CSEG
 start:

    ; 1. Carrega A e B inteiros nos registradores
    LDS R16, A ; AL
    LDS R17, B ; BL
    ADD R16, R17 ;AL + BL
    STS C, R16; CL = AL+ BL

    LDS R16, A+1 ; AL
    LDS R17, B+1 ; BL
    ADC R16, R17 ;AL + BL
    STS C+1, R16; CL = AL+ BL

    LDS R16, A+2 ;
    LDS R17, B+2 ;
    ADC R16, R17 ;
    STS C+2, R16;

    LDS R16, A+3 ;
    LDS R17, B+3 ;
    ADC R16, R17 ;
    STS C+3, R16;
```

```
    rjmp start
```

INDIRETO SEM INCREMENTAR

```
.DSEG
.ORG SRAM_START

A: .BYTE 4
B: .BYTE 4
C: .BYTE 4


.CSEG
 start:

   ; 1. Carrego pornteiro x com o endereço de a e assim sucessivamente
  LDI XL, LOW (A)
  LDI XH, HIGH(A)

  LDI YL, LOW(B)
  LDI YH, HIGH(B)

  LDI ZL, LOW(C)
  LDI ZH, HIGH(C)

  ;Agora soma dos bits, o zero

  LD R16, X
  LD R17, Y
  ADD R16, R17
  ST Z, R16

  ;Agora do bit 1

  ADIW XL, 1
  ADIW YL, 1
  ADIW ZL, 1
  LD R16, X
  LD R17, Y
  ADC R16, R17
  ST Z, R16

  ;Agora bit 2

  ADIW XL, 1
  ADIW YL, 1
  ADIW ZL, 1
  LD R16, X
```

```
  LD R17, Y
  ADC R16, R17
  ST Z, R16

  ;Agora o bit 3

  ADIW XL, 1
  ADIW YL, 1
  ADIW ZL, 1
  LD R16, X
  LD R17, Y
  ADC R16, R17
  ST Z, R16

   rjmp start
```

indireto com pós incremento

```
.DSEG
.ORG SRAM_START

A: .BYTE 4
B: .BYTE 4
C: .BYTE 4


.CSEG
 start:

   ; 1. Carrego pornteiro x com o endereço de a e assim sucessivamente
  LDI XL, LOW (A)
  LDI XH, HIGH(A)

  LDI YL, LOW(B)
  LDI YH, HIGH(B)

  LDI ZL, LOW(C)
  LDI ZH, HIGH(C)

  ;Agora soma dos bits, o zero

  LD R16, X
  LD R17, Y
  ADD R16, R17
  ST Z, R16

  ;Agora do bit 1
```

```
  LD R16, X+
  LD R17, Y+
  ADC R16, R17
  ST Z+, R16

  ;Agora bit 2

  LD R16, X+
  LD R17, Y+
  ADC R16, R17
  ST Z+, R16

  ;Agora o bit 3

  LD R16, X+
  LD R17, Y+
  ADC R16, R17
  ST Z+, R16

   rjmp start
```

INDIRETO COM DESLOCAMENTO:

```
.include "m328Pdef.inc"

.DSEG
.ORG SRAM_START
A: .BYTE 4
B: .BYTE 4
C: .BYTE 4

.CSEG
.ORG 0x0000

start_deslocamento:
; X NÃO SUPÓRTA ESSE, NÃO FAZ CONTAS DESSE TAMANHO
;então usasse ponteiros para apontar ao a e b (y e z)
; e usa o x para pegar o c que são só somas

   ; Aponta Y para a variável A (permanecerá fixo em 0x0100)
   LDI YL, LOW(A)
   LDI YH, HIGH(A)

   ; Aponta Z para a variável B (permanecerá fixo em 0x0104)
   LDI ZL, LOW(B)
   LDI ZH, HIGH(B)

   ; --- Byte 0 ---
```

```
    LDD R16, Y+0   ; Lê A
    LDD R17, Z+0   ; Lê B
    ADD R16, R17
    STS C, R16     ; Usamos STS para guardar o resultado

    ; --- Byte 1 ---
    LDD R16, Y+1   ; Lê A+1
    LDD R17, Z+1   ; Lê B+1
    ADC R16, R17
    STS C+1, R16

    ; --- Byte 2 ---
    LDD R16, Y+2   ; Lê A+2
    LDD R17, Z+2   ; Lê B+2
    ADC R16, R17
    STS C+2, R16

    ; --- Byte 3 ---
    LDD R16, Y+3   ; Lê A+3
    LDD R17, Z+3   ; Lê B+3
    ADC R16, R17
    STS C+3, R16

loop4: RJMP loop4
```